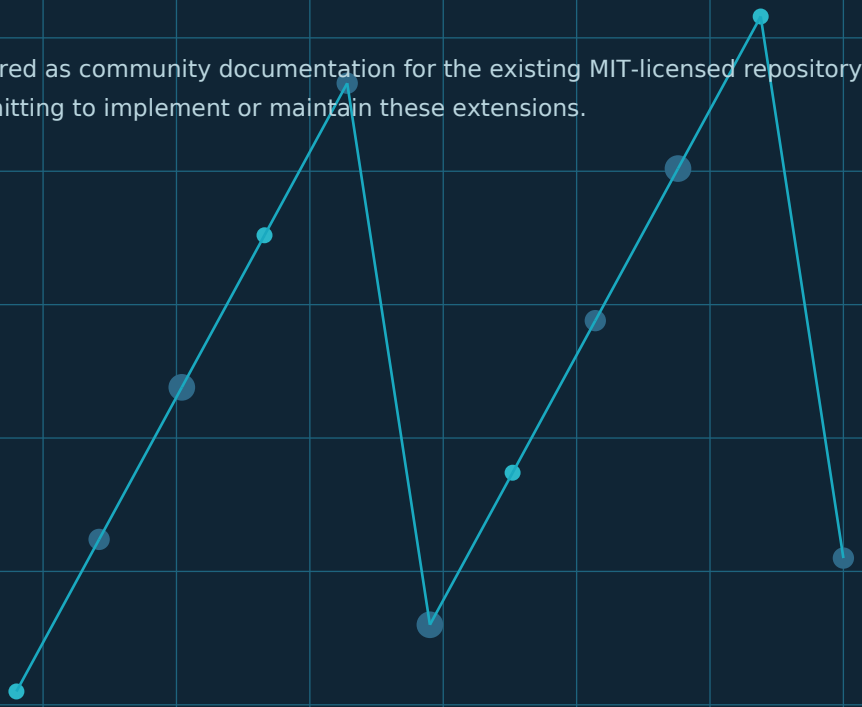
ARCHITECTURE / CONTRIBUTOR REFERENCE

Layer 1+ Neural World

Future Development Guide

One experimental neural core.
Four possible simulation scenarios.
Optional pathways to game-engine clients.

Prepared as community documentation for the existing MIT-licensed repository. The project author is not committing to implement or maintain these extensions.



PROPOSED TECHNICAL DESIGN | OCTOBER 2026

1 Purpose, scope and status

This guide turns a detailed, Danish-language extension proposal into an English, GitHub-friendly technical reference. It is intended for developers who wish to fork or independently extend the published Layer 1+ artificial-life experiment. The document deliberately does not add features to the released application.

> Release boundary. Only the original Layer 1 / 1+ experiment is implemented in the public repository. The Village, Nature, Colony, Robot and Game Engine Bridge designs below are proposals and illustrative code sketches, not tested modules or production-ready integrations.

What exists in the released project

Area	Current public implementation
Runtime	Node.js / TypeScript server; browser Canvas 2D viewer; local WebSocket snapshots.
Agent policy	Hand-written Float32Array neural network; 40 sensor inputs; two 48-unit hidden layers; 11 possible actions.
Learning and evolution	A2C-style policy/value learning with GAE, Adam; inheritance and mutation of genome and network weights.
World	Toroidal 2D arena with food, hazards, mirrors, energy, simulated affect, interaction and reproduction.
Optional cognition	Local Ollama for generated agent text; not the controller of the policy network.
State / exposure	In-memory simulation; no world persistence; local administrative controls are not authenticated.

How to read this guide

- Specified: values, structures and examples taken from the submitted extension proposal.
- Verified against repository: only basic existing paths, language/runtime, 40/48/11 dimensions and current WebSocket route.
- Design review: additional engineering cautions and suggested checks, clearly identified as recommendations.
- No hardware or performance claims: none of the proposed extensions has been benchmarked or tested by producing this PDF.

Contents

2 Core architecture and Scenario contract

3 Village simulation

4 Nature / wildlife simulation

5 Colony and symbolic signaling

6 Robot / warehouse world

7 Visual renderers and game-engine bridge

8 Suggested implementation sequence

9 Validation, safety and operational constraints

10 Source-sketch readiness review

11 Contribution and publication guidance

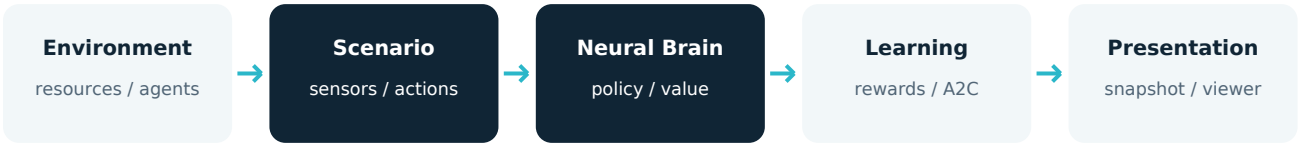
Appendix A Contracts and protocol examples

Appendix B Traceability and references

2 Core architecture: interchangeable scenarios

The central proposal is to separate the environment-specific state and rewards from the reusable policy learner. A scenario defines what an agent senses, which actions it can take, how a world tick changes state and which snapshots reach the viewer. The design proposes that A2C-style learning, inheritance, curiosity and optional Ollama narration remain reusable across variants.

PROPOSED EXTENSION PIPELINE - the existing learner remains a separate, reusable component



Scenario contract (conceptual)

```
interface Scenario {
  readonly cfg: ScenarioConfig;
  readonly t: number;
  paused: boolean;
  readonly inputDim: number;
  readonly numActions: number;
  readonly hiddenDim: number;
  readonly actionNames: readonly string[];
  sense(agent: ScenarioAgent): Float32Array;
  step(dt: number, onEvent: (event: ScenarioEvent) => void): ScenarioAgent[];
  agents(): ScenarioAgent[];
  snapshot(): ScenarioSnapshot;
  reset(): void;
  setTraceAgent(id: number | null): void;
  getBrainTrace(id: number): BrainTrace | null;
}
```

The proposed companion types are ScenarioAgent (including common body, policy, bonds and scenario-specific `data`), ScenarioConfig (world dimensions, population cap, runtime limit and parameters), ScenarioEvent and ScenarioSnapshot (agents, entities, events and statistics). The submitted proposal uses the identifiers `village`, `nature`, `colony` and `robot`.

Reusable vs. scenario-owned concerns

Keep shared	Implement per scenario
Neural network, value/policy heads, optimizer	Sensor layout and input normalization
Training loop design and experience buffer	Action names and transition rules
Agent / trace representation	Reward shaping and outcome events
Optional local LLM client contract	Resource entities, topology and UI-specific data
Transport and snapshot envelope	Scenario-specific scene/render mapping

> Compatibility requirement. Neural weights are tied to their input and output dimensions. A 40-input / 11-action baseline policy cannot be loaded directly into the proposed 64-input / 15-action Village model or 220-input Robot model. Add explicit versioning and a conversion policy if persistence is introduced.

3 Village: resources, roles and buildings

The submitted Village sketch is the most elaborate sample. It introduces a multi-resource economy, specialized work roles, construction sites, inventories, social behavior, trading and common storage while retaining the same general neural-policy pattern.

Entities and state

Component	Proposed model
Resources	food, wood, stone, gold; carried in an Inventory.
Roles	farmer, builder, guard, trader; roleAffinity and roleExperience per agent.
Buildings	house, farm, wall, tower, market; owner, current/max HP, built flag and progress.
Shared storage	Warehouse inventories for all four resource types.
Population	Agents have energy, integrity, social bonds, learned brain parameters and a scenario-specific data object.
World initialization	The population comes from initialPopulation in scenario config; the sketch seeds 80 food, 40 trees, 20 stones and 3 building sites.

Village policy dimensions

The proposal declares 64 input signals, 64 hidden units and 15 actions. The action list is north, south, east, west, rest, forage, chop_wood, mine_stone, build, trade, guard, social, court, deposit and withdraw.

Sensor indices	Information represented
0-9	Own state, movement, carrying capacity and reproduction cooldown.
10-15	Role encoding, role affinity and role experience.
16-19	Four inventory resource quantities.
20-39	Nearest food, tree, stone, unfinished site and finished building (4 channels each).
40-49	Nearest neighbor (6), local threat (4).
50-53	Shared warehouse quantities.
54-63	Curiosity (3), time/day cycle (3), population/building/neighbors/day-night (4).

Reward and specialization

A role-dependent multiplier modifies reward for work actions. The Farmer emphasizes food collection (forage 1.5); Builder emphasizes construction (build 1.8); Guard emphasizes patrol (guard 1.6); and Trader emphasizes trade (trade 1.8). The illustrative code updates role experience using pleasure and allows role changes after an age threshold.

> Design review. Do not assume that multiplying rewards by a predetermined role leads to emergent specialization. Measure role persistence, work completed, resource flow and reward hacking. The submitted role-switch snippet also contains a role-affinity calculation requiring correction before use.

Suggested acceptance checks

- No resource creation or double-spending through inventory/warehouse actions.
- Construction consumes the correct resource amounts and cannot complete repeatedly.
- Agents at max population cannot create unlimited newborns.
- All 64 sensor values are finite; action ordering matches network output size.
- Deterministic fixtures test trading, inheritance, role change and warehouse balance.

4 Nature: species and habitats

Nature is proposed as a closer relative of the original survival sandbox, with different species profiles and a simple habitat grid. Animals or plants would receive species-specific sensors, actions and rewards while reusing the generic neural-network implementation.

Species profiles specified in the source

Species	Inputs	Actions	Target	Reproduction	Energy cap
Plant	16	4	None	Asexual / litter 2	50
Herbivore	48	9	Plant	Sexual / litter 2	120
Carnivore	56	11	Prey	Sexual / litter 1	160
Scavenger	44	8	Carcass	Sexual / litter 3	100

The example assigns reproduction cooldowns of 60, 30, 45 and 25 seconds respectively. Metabolic multipliers are 0.1, 1.0, 1.4 and 0.8. These are proposed tuning values, not validated ecological parameters.

Habitat model

A grid class maps coordinates into water, desert, grassland, forest or mountain, with a habitat-fitness lookup for every species. The source sketch uses a deliberately simple trigonometric/random biome generator and recommends replacing it with a more suitable noise method in a fuller implementation.

Biome	Plant	Herbivore	Carnivore	Scavenger
Water	0.20	0.10	0.40	0.30
Desert	0.05	0.10	0.30	0.50
Grassland	0.90	1.00	0.60	0.70
Forest	0.70	0.90	0.90	0.60
Mountain	0.20	0.30	0.70	0.50

> Important distinction. Although all species can share the same Brain class, they cannot share weight matrices directly when input/output dimensions differ. Cross-species mutation, migration, or inheritance needs a specifically designed compatible representation.

Tests developers should add

- Fixed-seed terrain and habitat repeatability; correct bounds when positions lie on grid edges.
- Valid energy and reproduction behavior by species across long simulated runs.
- Non-zero survival resources and no impossible habitat spawn loops.
- Metrics such as population stability, predation, extinction and trait diversity, rather than visual impressions alone.

5 Colony: tasks and communication

The Colony proposal extends settlement simulation with assigned tasks and a short-lived symbolic signaling layer. The source describes task queues and a 16-symbol communication alphabet; it does not demonstrate an emergent language.

TaskQueue proposal

Tasks may be `build_wall`, `dig_tunnel`, `gather_ore`, `repair_generator` or `defend`. Each holds a position, required/current work, priority, assignment and creation timestamp. The illustrative selection heuristic scores a task as priority × 100 − distance.

Signaling proposal

Field	Meaning
symbol	Discrete identifier 0-15; initially without predefined semantics.
agentId / pos	Sender and emission location.
t / ttl	Emission time and time-to-live; default TTL shown as 2 seconds.
sense()	Number of emissions nearby, nearest symbol, distance and a placeholder similarity field.
coordinationReward()	Illustrative +0.05 for matching recent symbols from another nearby agent during a short window.

The proposed code caps stored emissions at 200 and searches within distance thresholds. Communication is entirely a proposed local simulation mechanism: it is not a built AI-to-AI networking feature, does not transmit optically and does not connect to ChromaNeural.

> Scientific caution. Rewarding equal symbols can produce symbol copying or collapse to one symbol, not meaningful language or successful task communication. Test against random-signal baselines and measure whether signals improve objective coordination.

Minimum tests

- TTL expiry with a simulation clock (not mixed Date.now() and simulation time).
- No unbounded memory growth, duplicate-task assignment or stale agent references.
- Compare coordination success with signaling disabled, randomized and learned.
- Verify messages cannot trigger external/network actions without explicit permissions.

6 Robot world: structured control tasks

The Robot sketch places agents in a warehouse-like grid where they observe a nearby 5×5 area and manage battery-powered tasks such as picking up, carrying, dropping off and charging.

Grid and neural input

Each cell is assigned one of eight kinds: `empty`, `wall`, `conveyor`, `shelf`, `package`, `drop_zone`, `charger` or `hazard`. A 5×5 one-hot neighborhood gives 25 × 8 = 200 values, followed by 20 self/neighbor/time values for a proposed 220-input neural policy.

Actions and agent-specific state

Category	Proposal
Actions (10)	north, south, east, west, wait, pickup, drop, charge, scan, yield.
Battery state	battery, maxBattery, lastChargeT.
Carrying state	carrying, carryingId, completedDeliveries, currentTarget.
Learning signals	Delivery success, charging decisions, collision/hazard avoidance; scanning is described as carrying an energy cost.

The grid code initializes shelves and charging stations in fixed locations. The broader world behavior, energy/battery drain, scanner range, package lifecycle and collisions are not fully implemented in the submitted sketch.

Engineering acceptance checks

- Robots cannot pass walls, pick up the same package twice or deliver nonexistent cargo.
- Battery never becomes negative and charging is only possible at allowed cells.
- Robot position in discrete cells and neural movement semantics are consistent.
- Sensor ordering stays stable and is documented with reproducible fixtures.
- A CPU budget and agent-cap check prevents runaway training on smaller hardware.

7 Rendering and game-engine connections

The proposal offers two independent presentation routes, both keeping the TypeScript simulation authoritative. A renderer should present snapshots; it should not silently redefine the physics or neural model.

Option A - richer browser visualization

The smallest visual step is to replace or supplement the Canvas 2D viewer with a richer browser renderer such as Three.js or PixiJS, subscribed to the existing local WebSocket at /ws. The illustration shows 2D positions translated to a 3D plane, creating one mesh per agent and updating it from snapshots.

```
// Illustrative client-side shape; adapt to current /ws envelope.
const ws = new WebSocket(`ws://${location.host}/ws`);
ws.onmessage = ({ data }) => {
  const message = JSON.parse(data);
  if (message.type !== 'state') return;
  const snapshot = message.payload;
  // Update or create visual entities using snapshot.agents.
};
```

> Porting note. The supplied example labels the browser file viewer3d.js but includes TypeScript-only syntax in one line. Use valid JavaScript or compile a separate TypeScript frontend. The renderer still needs object removal, asset loading, interpolation and scene lifetime management.

Option B - external game engine

A proposed `EngineBridge` runs as a WebSocket adapter between the authoritative TypeScript process and a client built with Unity, Godot or potentially Unreal Engine. The code sketch includes separate protocol definitions and example Unity C# and Godot GDScript clients.

Direction	Suggested envelope / messages
Server → engine	snapshot (tick, agents, entities), event, config.
Engine → server	input (agent action request), spawn, event, query.
Node implementation	ws-based EngineBridge broadcasting snapshots at an illustrative 20 Hz.
Unity sample	NativeWebSocket client; prefab creation and transform updates.
Godot sample	WebSocketPeer client; scene instantiation and cleanup.

Conceptual JSON snapshot

```
{
  "type": "snapshot",
  "tick": 120,
  "agents": [{
    "id": 17, "name": "Agent-17",
    "pos": { "x": 4.2, "y": 7.3 },
    "rot": 0.9, "energy": 62, "action": "forage"
  }],
  "entities": [{ "id": 81, "kind": "food", "pos": { "x": 8, "y": 2 } }]
}
```

This is an explanatory example, not the current public server message format. Its fields and coordinates must be mapped and versioned before integrating any engine.

Bridge requirements before public use

- Bind to loopback by default, authenticate authorized clients, validate every message and reject unrecognized actions.
- Add schema version, client capability negotiation, message size limits and rate limits.
- Never let arbitrary game-client input override agent policy until explicit permission and behavior semantics are designed.
- Use deterministic simulation time; define tick ordering, interpolation and reconnection recovery.
- Document per-engine version/dependency constraints and avoid assuming proposed sample code compiles unchanged.

8 Implementation order suggested by the source

The original proposal concludes with the following order, intentionally different from the order of its presentation: refactor the baseline first, then build incrementally from relatively small to complex changes.

Stage	Suggested work	Acceptance gate
0. Baseline	Freeze original version in a fork/branch; record reproducible behavior.	Baseline untouched; rollback available.
1. Scenario core	Extract Scenario interface; keep the original as a basic scenario.	Same visible behavior; existing tests still pass.
2. Colony	Add TaskQueue and local symbolic signal experiments.	Task lifecycle, TTL and coordination tests.
3. Robot	Add grid, battery and structured delivery tasks.	Valid 220D sensors; collision and delivery invariants.
4. Nature	Add distinct species profiles and habitat grid.	Per-species dynamics, controlled population tests.
5. Village	Add the full role/resource/trade/building economy.	Resource conservation and long-run stability checks.
6. Engine bridge	Expose versioned state to richer renderers / external clients.	Schema, security, reconnect and performance tests.

Each stage is optional and can be implemented independently. This sequence is offered for outside contributors and does not constitute a release commitment from the repository owner.

Preserve easy exits

- Use a feature branch or fork for every scenario.
- Keep the current baseline available rather than renaming or replacing its files in place.
- Allow the old Canvas viewer to continue consuming the original snapshot contract.
- Publish verification and benchmark data per scenario, and keep test claims proportional to actual results.

9 Validation, security and runtime limits

Validation matrix for contributors

Category	Suggested minimum evidence
Static	TypeScript typecheck, lint and explicit schema/version checks.
Neural	Fixed-seed network shape, finite outputs, gradient stability and reward invariants.
Simulation	Controlled ticks, survival/resource conservation and entity lifecycle.
Protocol	Malformed JSON, unknown types, oversized payloads, untrusted input and reconnect behavior.
Rendering	Agent creation/update/removal, coordinate mapping, scene cleanup and frame budget.
Behavior	Ablation tests and metric baselines, not just visually plausible output.
Security	Local bind defaults, authentication, origin controls and input sanitization.
Performance	CPU usage and temperature over realistic intervals; population and model limits.

Runtime and CPU caution

The current handcrafted policy learner runs on the CPU in TypeScript; a GPU is not required by the baseline neural arithmetic, and adding one does not automatically accelerate it. An optional Ollama language model can add CPU/GPU load depending on how it is configured. For constrained PCs, cap population and tick frequency, disable optional LLM calls, and monitor heat rather than relying only on the runtime guard.

> Security boundary. The public Layer 1+ experiment is intended for trusted local use and its administrative endpoints are not authenticated. The proposed EngineBridge likewise contains no production authentication or authorization. Do not expose either to the public Internet as written.

Experimental claims

Behavior such as bonding, curiosity, simulated pleasure, a mirror-mark task or matching symbols is a property of the implemented reward and state rules. It does not, by itself, validate self-awareness, consciousness, biological evolution or meaningful communication.

10 Readiness review of the supplied code sketches

The submitted 1,800-line source notes are valuable design references but not a compiled or tested patch set. The following are concrete integration issues found by reading the examples and comparing selected baseline interfaces; they should be handled by anyone implementing the ideas.

Observed issue	Required follow-up
Missing data in generic agent	Proposed ScenarioAgent does not declare curiosity, though Village sensors use agent.curiosity.
Clamp signature mismatch	Proposed clamp(v,a,b) is invoked with only one argument in several sensor mappings.
Village role-affinity calculation	bestScore minus exp[bestRole] becomes zero for the selected role; revisit transition formula.
Different Brain sizes	Village 64/15, Nature species-specific, Robot 220/10 require separate network shapes and saved-weight contracts.
Unchecked world/agent integration	Scenario adapters are not wired into the current src/index.ts and src/world/world.ts baseline.
Bridge input is not active	overrideInput is a placeholder; spawn/event examples lack an authoritative implementation.
Scene snapshot mismatches	Bridge rotation refers to vx/vy fields that are not present in the proposed Village snapshot.
Language/runtime mismatch	Browser viewer3d.js sample contains TypeScript casting; it needs a build step or JS rewrite.
Rendering formats	Unity example parses HTML-style colors while proposed agent colors may be hsl(...).
Godot and Unity samples	Client libraries, engine versions and incomplete entity update logic require testing.
Temporal control	Some proposed signaling/robot data uses Date.now rather than simulation time.
Scientific interpretation	Same-symbol reward can collapse to trivial signaling rather than shared semantics.

> Result of this review. The guide is suitable as an idea and architecture reference. It should not be advertised as a working Village, Nature, Colony, Robot or Unity/Godot implementation. No compilation or runtime execution of those proposed extensions was performed.

11 Contributor guidance and publication

Contributors may independently fork the repository under its MIT license and adapt the code. The project owner is sharing an experimental baseline and extension ideas rather than promising future levels.

Suggested change-package format

- Start with a goal and an observable behavioral hypothesis.
- Reference the original baseline commit and describe altered sensors/actions/rewards.
- Include architecture and protocol version changes where relevant.
- Add tests, validation instructions, measured outcomes and clearly marked unverified portions.
- Keep patches small and leave unrelated TypeScript source, server controls and baseline documentation untouched.

- Document graphics provenance, third-party dependency licenses and attribution.

What a contribution should not claim

- That the illustrations are screenshots of existing features.
- That the supplied sketches are guaranteed to typecheck or integrate without adaptation.
- That symbolic signaling proves a new language or consciousness.
- That external ChromaNeural, PRISME or physical optical hardware is already linked to this repository.
- That a personal or simulated performance observation establishes verified hardware capability.

Relationship to possible wider AI communication

An independent future project might use a safe simulation as a testing arena for AI-to-AI interactions. That is an optional research direction only. The real-world ChromaNeural software, any LAN/WAN networking, color/shade optical signaling and physical validation remain separate projects and are not dependencies or built features of Layer 1+.

Appendix A Illustrative contracts

The short extracts below retain the architecture and terminology of the submitted proposal, with English comments and smaller examples for readability. They are sketches; references to unimplemented classes and modules intentionally remain unresolved.

A1. Scenario configuration and snapshot

```
type ScenarioName = 'village' | 'nature' | 'colony' | 'robot';
interface ScenarioConfig {
  name: ScenarioName;
  worldWidth: number; worldHeight: number;
  initialPopulation: number; maxPopulation: number;
  maxRuntimeMinutes: number;
  params: Record<string, number>;
}
interface ScenarioSnapshot {
  t: number; paused: boolean; w: number; h: number;
  agents: Array<Record<string, unknown>>;
  entities: Array<Record<string, unknown>>;
  events: ScenarioEvent[];
  stats: Record<string, unknown>;
}
```

A2. Village inventory, roles and actions

```
interface Inventory { food: number; wood: number; stone: number; gold: number }
type Role = 'farmer' | 'builder' | 'guard' | 'trader';
const VILLAGE_ACTIONS = [
  'north','south','east','west','rest','forage','chop_wood',
  'mine_stone','build','trade','guard','social','court',
  'deposit','withdraw'
] as const;
```

A3. Colony signaling: proposed data exchange

```
interface SymbolEmission {
  agentId: number;
  pos: { x: number; y: number };
  symbol: number;  // 0..15, with no predefined meaning
  t: number;       // define a consistent simulation clock
  ttl: number;     // seconds
}
```

A4. Robot grid observation

```
type CellKind =
  | 'empty' | 'wall' | 'conveyor' | 'shelf'
  | 'package' | 'drop_zone' | 'charger' | 'hazard';
const LOCAL_CELLS = 5 * 5;
const ONE_HOT_CATEGORIES = 8;
const ROBOT_INPUT_DIM = LOCAL_CELLS * ONE_HOT_CATEGORIES + 20; // 220
```

A5. Engine bridge message taxonomy

```
type ServerToEngine =
  | { type: 'snapshot'; tick: number; agents: unknown[]; entities: unknown[] }
  | { type: 'event'; event: { kind: string; text: string } }
  | { type: 'config'; config: Record<string, unknown> };
type EngineToServer =
  | { type: 'input'; agentId: number; action: string }
  | { type: 'spawn'; agentType: string; pos: { x: number; y: number } }
  | { type: 'event'; text: string; agentId?: number }
  | { type: 'query'; query: 'agents' | 'world' | 'stats' };
```

A production protocol would require authentication and a strict validation layer before accepting the EngineToServer requests.

Appendix B Traceability and references

Source alignment

Part	Origin / confidence
Original implementation facts	Checked against public repository src/world/types.ts, src/neural/network.ts, src/index.ts and src/world/agent.ts.
Extension interfaces and scenario parameters	From the supplied Danish-language development proposal (11 October 2026); proposed, not implemented.
Migration sequence	From the proposal closing recommendations, preserved in the supplied order.
Readiness cautions	Editorial review of the provided examples and relevant existing interfaces; not a full code audit.
Illustrations	The project README has separate, explicitly conceptual artwork; none is presented as a built feature.

Repository locations

https://github.com/Janus5G/layer1-neural-world

README.md - public project status and setup

docs/ARCHITECTURE.md - current baseline architecture

docs/EXTENDING.md - existing general contributor guidance

docs/ROADMAP.md - original longer-range ideas

docs/ETHICS.md - interpretation and responsible experimentation

> Final status. This PDF documents possible future work for external developers. It is not a new release, does not modify the public software, and does not create an obligation for the original author to pursue any scenario.

Document version 1.0 | 11 October 2026 | English | Layer 1+ Neural World